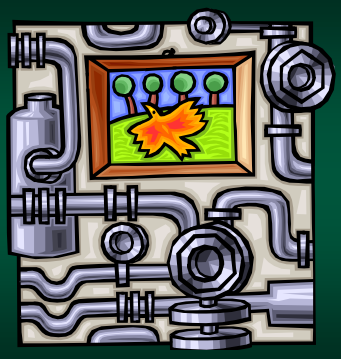
Stacks & Queues

Part 3

1



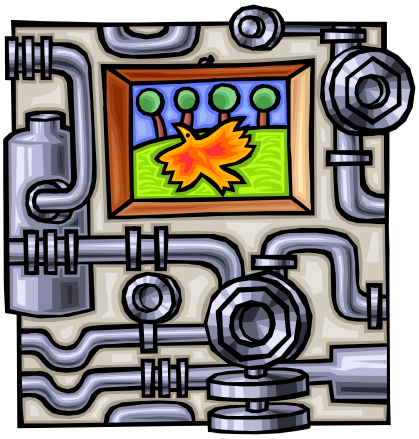
Data Abstraction

Abstraction is power

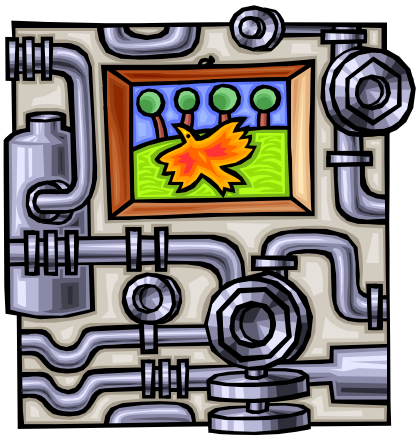
2

Abstract Data Types

- *Data types* are used in practically all programming languages
- The core data types found in language is known as a *primitive data type*



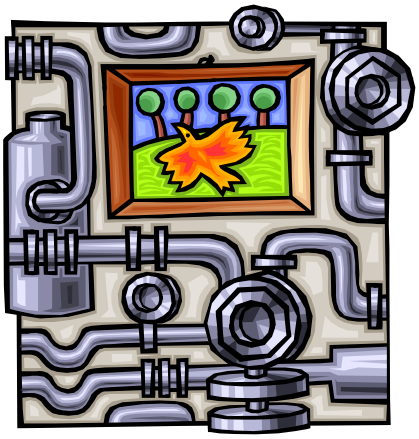
Abstract Data Types



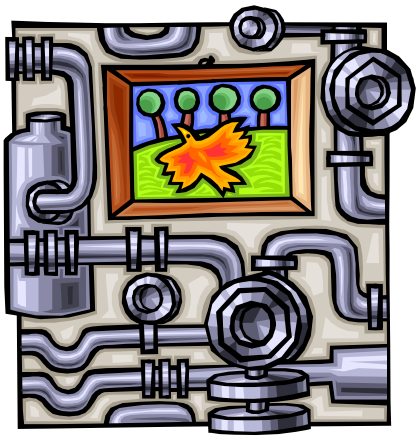
- An *abstract data type (ADT)* hides how it is implemented from the *client* (programmer)
- The client only interacts with the defined operations

Abstract Data Types Specify 3 Things

1. Set of possible values
2. *Operations* on the data
   - alternatively called *functions*
   - ... or *methods*
3. The errors can occur during each operation

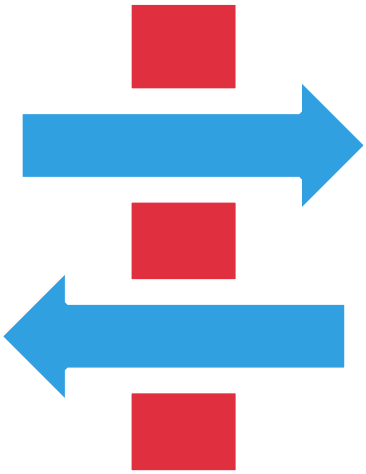


Abstract Data Types



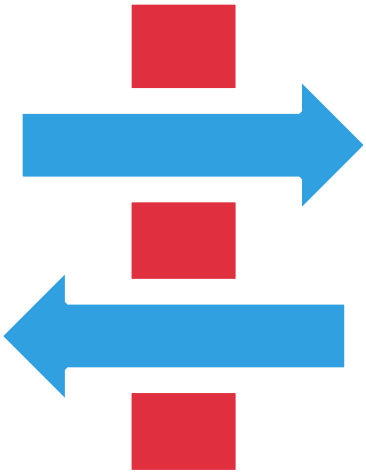
- This layer of abstraction separates implementation from behavior
- And, it allows you to change the data structure – without breaking the ADT

ADTs vs Data Structures



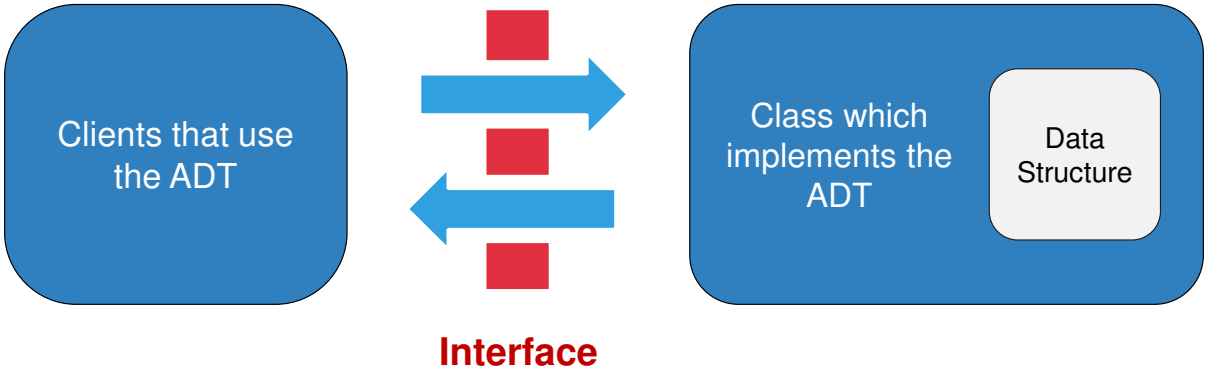
- An ADT is implementation independent
- Can, internally, use any data structure
  - array, linked list, etc…
  - depending how the ADT works, some are better than others

ADTs vs Data Structures



- ADT defines an *interface*
- It defines:
  - operations (public methods)
  - properties (public fields)

Data Structures



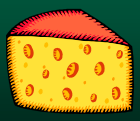
Example ADT: Cheese Trader

- Data stores orders of cheese
- The operations supported are
  - buy (cheese, count)
  - sell (cheese, count)
  - cancel (Order)
  - balance – current funds

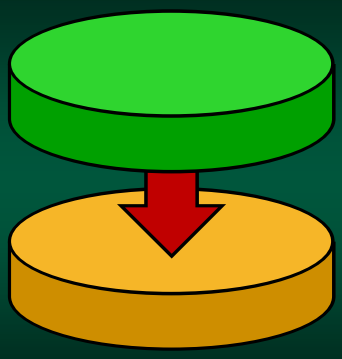
Example ADT: Cheese Trader

- Error conditions:
  - nonexistent cheese
  - sell a cheese we don't have
  - count is not greater than 0

Cheese Trader Interface



class CheeseTrader		
int	buy(String name, int count)	*Returns order #*
int	sell(String name, int count)	*Returns order #*
void	cancel(int order)	
double	balance()	



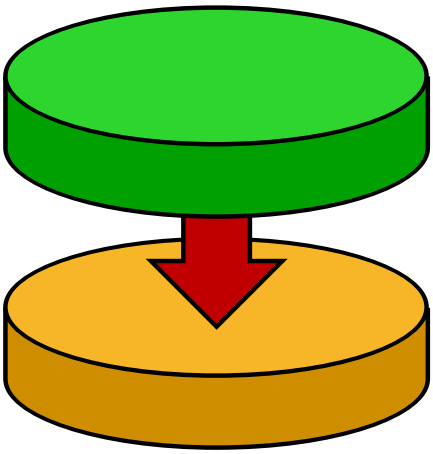
Stacks

Piles of… Data

13

Stack

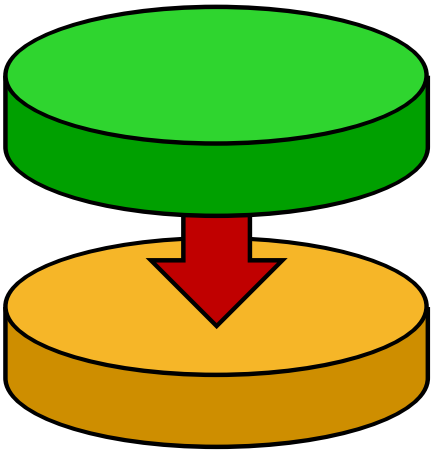
- The *Stack ADT* stores objects based on the concept of a stack of items – like a stack of dishes
- Data can only be added to or removed from the top of the stack



14

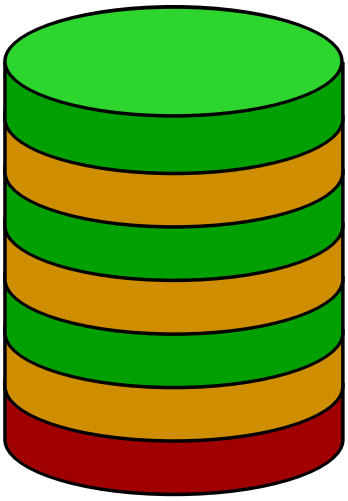
Stack

- This gives a first-in-last-out logic (aka FILO)
- Same concept is also called last-in-first-out (LIFO)



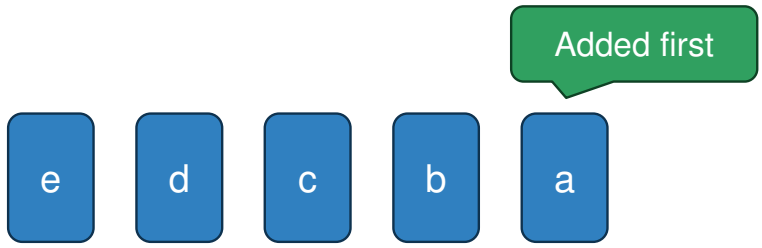
Stack Operation: Push

- A value is added to the stack
- It is placed on the top location
- Rest of the items are "covered"



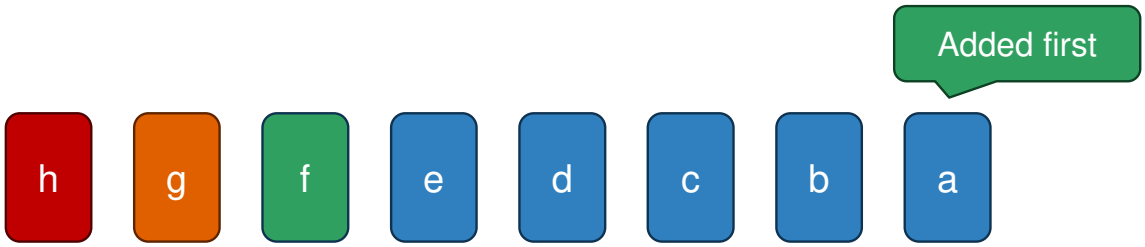
Push Location

- Pushed items are added at the front of a list
- So, any pushed item is placed at the head



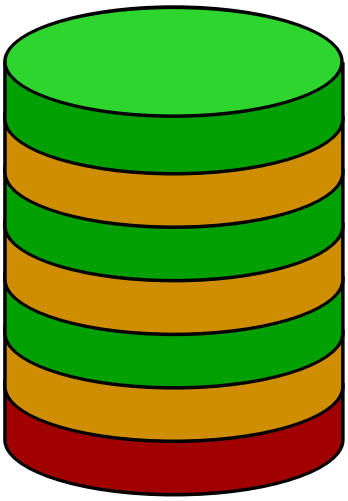
Push Location

- Pushed items are added at the front of a list
- So, any pushed item is placed at the head



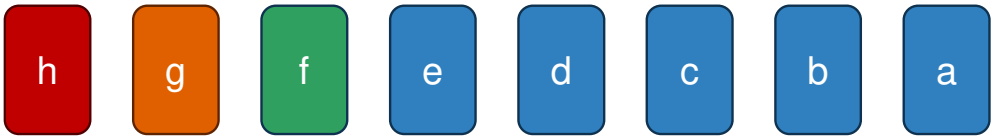
Stack Operation: Pop

- Removes an item from the stack
- Last item added is removed
- 2nd item becomes the top



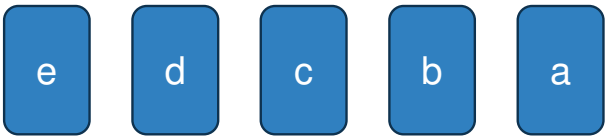
Pop Location

- Popped items are removed from the front of a list
- So, any popped item is taken from the head



Pop Location

- Popped items are removed from the front of a list
- So, any popped item is taken from the head



Stack Interface

class Stack		
void	push(object item)	
object	pop()	
object	peek()	*Return the top. Sometimes called top()*
boolean	isEmpty()	

Auxiliary Operations

Stacks: Error Conditions

- The execution of an operation may sometimes cause an error condition, called an *exception*
- Exceptions are said to be “thrown” by an operation that cannot be executed
- In the Stack ADT, operations pop and top cannot be performed if the stack is empty

Resizing an Array-Based Stack

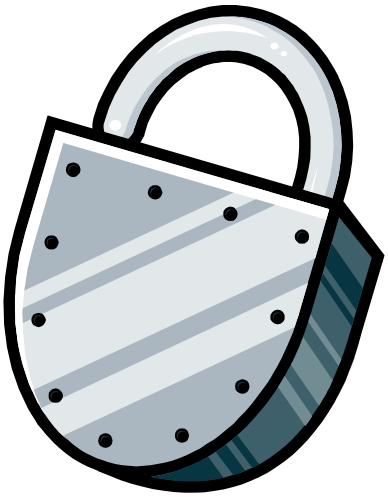
- For stacks, if a dynamically allocated array is used, each pop/push will require the entire array to be resized
- It will require O(n)
- So, a dynamic array is a poor choice

One Solution… Not a Great One

- We could use an ArrayList
- So, the array will resize only when a new "block" of elements is needed
- Like a fixed-capacity array, we need to keep an end index

Fixed-Capacity Stacks

- A fixed-capacity array can be used instead
- For a *fixed-capacity stack*, an array is an excellent choice – in specific situations…



Array-Based Fixed-Capacity Stack

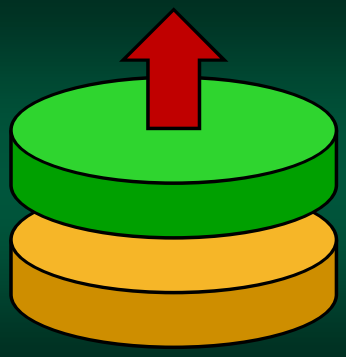
- The stack would behave as normal until the capacity is reached
- In this case, one of two things will happen…

When the Stack is filled…

1. Stack throws an *Overflow Error*
2. Stack discards an object
   - the bottom of the stack is typically removed
   - this gives the space needed for the newly pushed object
   - e.g. the history feature of your web browser

Stack Summary

Operation	Fixed-Capacity Array	Dynamically Allocated Array	Singly Linked List
Pop()	O(1)	**O(n)**	**O(1)**
Push()	O(1)	**O(n)**	**O(1)**
Peek()	O(1)	O(1)	**O(1)**

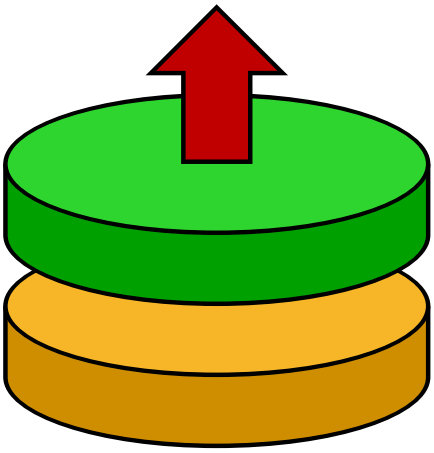


Queues

Conga-line of Data!

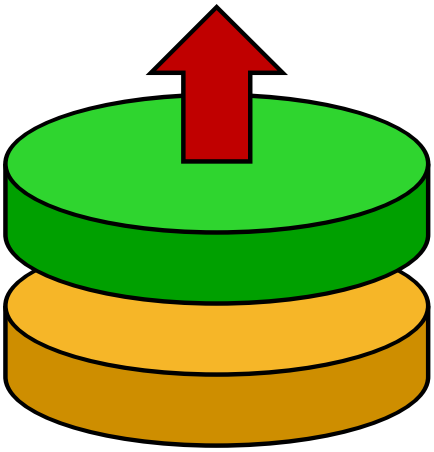
Queues

- *Queue ADT* stores list of arbitrary objects
- Based on the concept of a line – e.g. when you buy groceries
- Objects enter the back of the line, and must wait for prior items to leave before they do



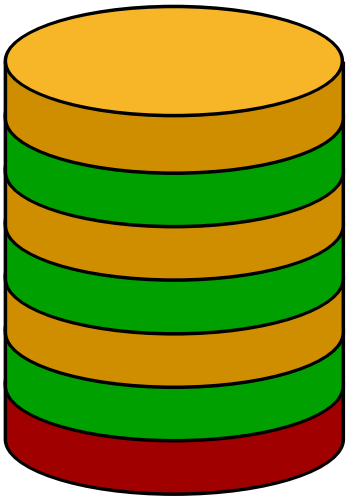
Queues

- In most parts of the World, they call a "line" a "queue"
- Main queue operations:
  - enqueue (object): place on item on the queue
  - dequeue: removes and returns the first inserted object



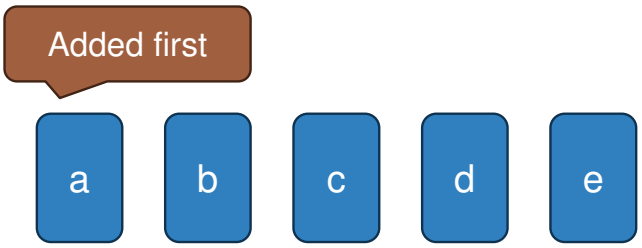
Queue Operation: Enqueue

- When an object is "enqueued", it is put on to the **end** of the queue
- The items on the top of the queue are not covered



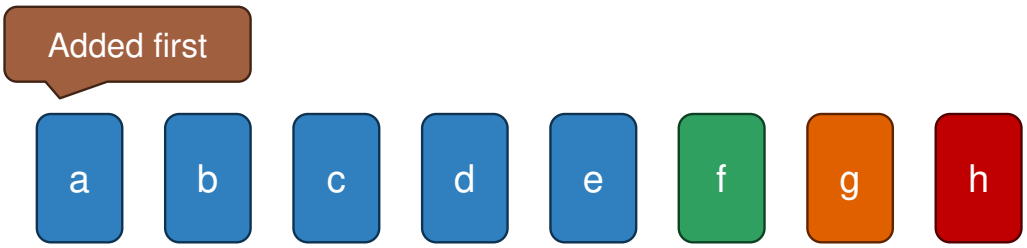
Enqueued Location

- Enqueued items are added at the end of a list
- So, any enqueued item is placed at the tail



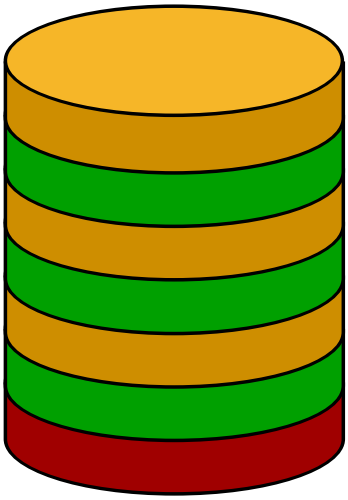
Enqueued Location

- Enqueued items are added at the end of a list
- So, any enqueued item is placed at the tail



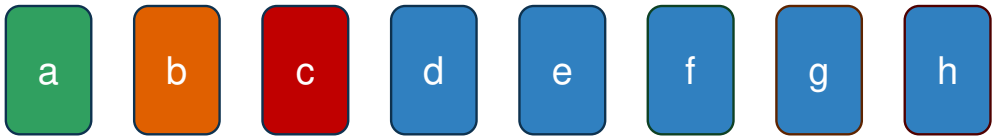
Queue Operation: Dequeue

- Dequeue removes the item from the front of the queue
- Second item becomes the new first item
- This gives a first-in-first-out logic (aka FIFO)



Dequeued Location

- Dequeued items are removed from the front of a list
- So, any dequeued item is taken from the head



Dequeued Location

- Dequeued items are removed from the front of a list
- So, any dequeued item is taken from the head

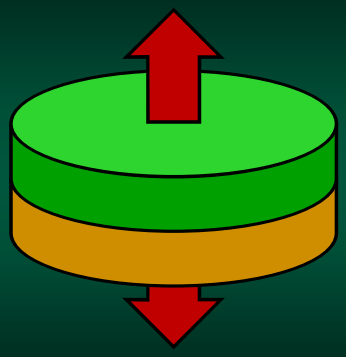
Queue Interface

```
public class Queue
    void     enqueue(object item)
  object     dequeue()
  object     peek()                    Return first item, without dequeue
 boolean     isEmpty()
```

Auxiliary Operations

Queue Summary

Operation	Fixed-Capacity Array	Dynamically Allocated Array	Singly Linked List
Enqueue()	O(1)	**O(n)**	**O(1)**
Dequeue()	O(1)	**O(n)**	**O(1)**
Peek()	O(1)	**O(1)**	**O(1)**



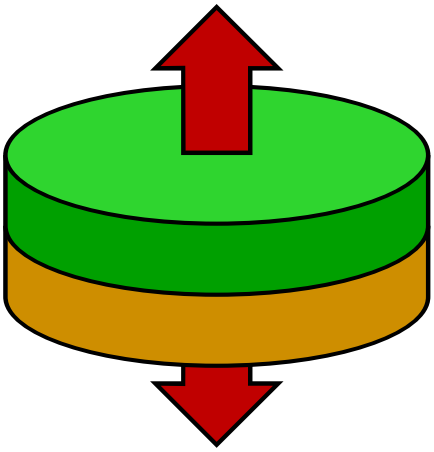
The Deque ADT

Time to shuffle the "deck"

41

Deque ADT

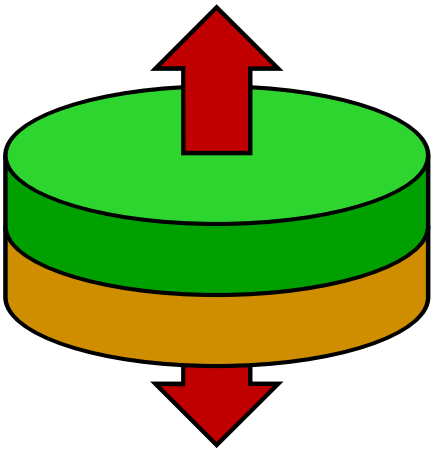
- There is a variant of the queue called a *deque* (pronounced "deck")
- The name is derived from **d**ouble-**e**nded **que**ue (sometimes it is shorted more to DQ)



42

Deque ADT

- As the name implies, it's a queue allows insertions and removals from both ends
- It is a merging of a stack and queue ADT and the operations are union of the two
- Be warned: name of each operation varies greatly between programming languages



Deque ADT

- addFront
  - place an object on the front of the deque
  - this is same as stack "push"
  - also called: offerFirst, pushFirst
- addBack
  - place an object on the end of the deque
  - this is the same as queue "enqueue"
  - also called: offerLast, pushLast

Deque ADT

- removeFront
  - remove an object from the front of the deque
  - same as: queue "dequeue" or stack "pop"
  - also called: pollFirst, popFront
- removeBack
  - this is unique – and not found in either a stack or queue ADT
  - also called pollLast, popBack

Deque Interface

class Deque	
void	addFront(object item)
void	addBack(object item)
object	removeFront()
object	removeBack()
object	peekFront()
object	peekBack()
boolean	isEmpty()

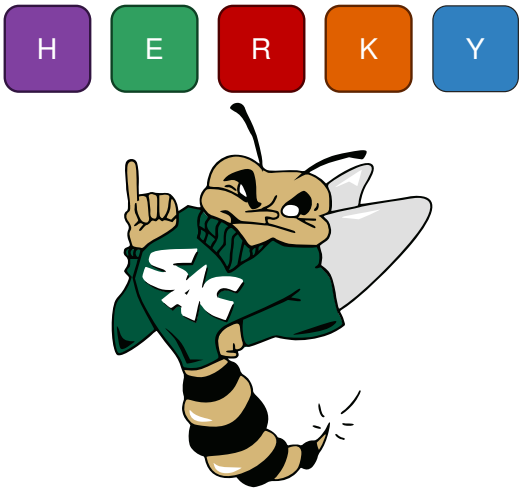
Auxiliary Operations

Deque Example

```
1. addFront('R')
2. addBack('K')
3. addFront('E')
4. addBack('Y')
5. addFront('H')
```

Deque Example

```
1. addFront('R')
2. addBack('K')
3. addFront('E')
4. addBack('Y')
5. addFront('H')
```



Deque Advantages

- A deque can function as either a stack or queue
- "Add Front" operation can be used to "redo" or "undo" a queue removal – remove then put it back in line
- There are some scenarios where this logic is needed

Deque Disadvantages

- Stacks/Queues can be implemented with a singly-linked-list
- *Deques require a double-linked-list*
  - otherwise, removing items from the end would require O(n) – *even with a tail node*
  - also, the link overhead (memory required) is <u>doubled</u>

Deque Summary

Operation	Fixed Capacity Array	Dynamically Allocated Array	Singly Linked List	Double Linked List
addFront()	O(1)	**O(n)**	**O(1)**	**O(1)**
addBack()	O(1)	**O(n)**	**O(1)**	**O(1)**
removeFront()	O(1)	**O(n)**	**O(1)**	**O(1)**
removeBack()	O(1)	**O(n)**	**O(n)**	**O(1)**

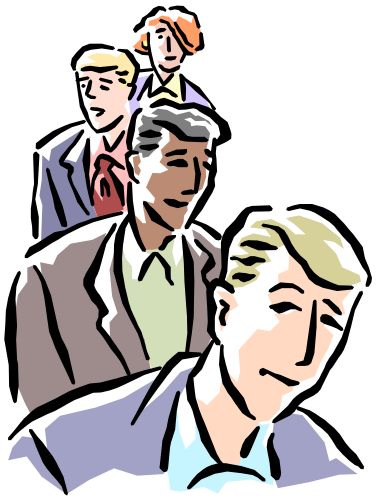
Queues & Stacks in Practice

1001 Uses!

(I meant 1,001 – not 9)

HTML Tag Matching

- HTML is a hierarchical structure
- HTML consists of tags
  - each tag can also embed other tags
  - allows text to be aligned, made bold, etc…



HTML Tag Matching

- Web browsers read the text and apply a tag depending if it is active
- They maintain a stack…
  - push a start tag, pop and end tag
  - if the HTML is correct, they should match
  - *… with the exception of the unary tags*

```
<html>
<body>
<center>
<h1>Banks of Sacramento</h1>
<i>A bully ship and a bully crew.<br>
Hoo-da! Hoo-da!<br>
A bully mate and a captain too.<br>
Hoo-da! Hoo-da-day!<br>
And it's blow, ye winds, blow,<br>
for Californi-o.<br>
For there's plenty of gold,<br>
so I've been told,<br>
on the banks of the Sacramento.</i><br>
</center>
</body>
</html>
```

HTML

Banks of Sacramento

A bully ship and a bully crew.
Hoo-da! Hoo-da!
A bully mate and a captain too.
Hoo-da! Hoo-da-day!

Then blow, ye winds, blow,
for Californi-o.
For there's plenty of gold,
so I've been told,
on the banks of the Sacramento.

Sea shanty from the Gold Rush days!

Balanced Parentheses

- When analyzing arithmetic expressions often the structure of the expression needs to be checked
- For example:
  - are operators in the correct place?
  - are the parenthesis balanced?

Balanced Parentheses

- Let's look at parenthesis
- One approach…
  - can we just use a "parenthesis count"
  - if it isn't `0` at the end then the expression is invalid
- Sorry, it won't work…
  - some expressions allow `{ }` and `[ ]`
  - …and they may be in the wrong place

Balanced Parentheses

- A great solution is a stack
- Approach…
  - push each (and pop each)
  - at the end, the stack should be empty
  - also, if you attempt to pop on an empty stack, the expression is invalid
- It can also catch mismatched symbols

Balanced Parenthesis Examples

`(a + b)`	Balanced
`(a + b))`	Pop empty stack
`) a + b (`	Pop empty stack
`(a + (b + 1) * c) / e`	Balanced
`(a * (b + ((d + e) * f))`	Stack has 1 left

Balanced Parenthesis Examples

`[a + b]`	Balanced
`(a + b}`	Mismatch
`{[ a + b }]`	Mismatch
`(a + (b + 1) * c / e`	Unbalanced
`(a * [b + {c + d} * e])`	Balanced

Evaluating Expressions

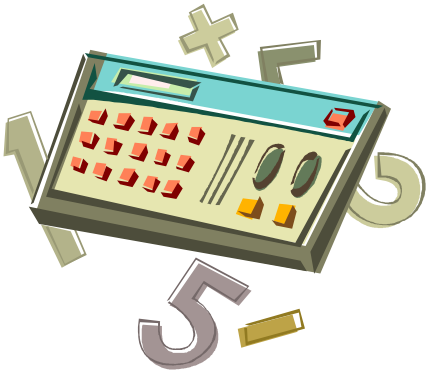
A Stack and Queue working together!

Evaluating Expressions

- It is a common task in programs to evaluate mathematical expressions and get a result
- Computers can perform this task using an algorithm *created by Dijkstra*, but we will get into that later

Evaluating Expressions

- First, we need to look at mathematical expressions
- We usually use infix notation
  - not stack or queue "friendly"
  - there are, however, two alternative notations
  - *one of which is stack friendly*



Infix Notation

- Using *infix notation*, we put the operator in between the two operands
- This is the standard format used today

To add the numbers *a* and *b*, we type:	`a + b`
To divide *a* by *b*, we type:	`a / b`

Prefix Notation

- *Prefix notation*, rather than putting the operator between the operands, puts it first
- It is also called *"Polish Notation"*
- Used by the LISP programming language

To add the numbers *a* and *b*, we type:	`+ a b`
To divide *a* by *b*, we type:	`/ a b`

Postfix Notation

- *Postfix notation* puts the operator at the end
- Also called *"Reverse Polish Notation" (RPN)*

To add the numbers *a* and *b*, we type:	`a b +`
To divide *a* by *b*, we type:	`a b /`

Postfix Notation

- Since the operator is last, we can also use it as a "trigger" to perform math
- In fact, classic Hewlett Packard calculators are stack based

To add the numbers *a* and *b*, we type:	`a b +`
To divide *a* by *b*, we type:	`a b /`

Where are My Parenthesis?

Infix	Prefix	Postfix
a + b * c	+ a * b c	a b c * +
(a – b) * c	– a b * c	a b – c *
(a / (b – c) + d)	+ / a – b c d	a b c – / d +
(a + b / (c – d))	+ a / b – c d	a b c d – / +

Where are My Parenthesis?

- Infix is the <u>only</u> notation that needs parentheses to change precedence
- The order of operators handles precedence in prefix and postfix

Compute Postfix Algorithm

- Computing a postfix expression is easy
- All you need is:
  - one input queue that contains values & operators
  - and one stack

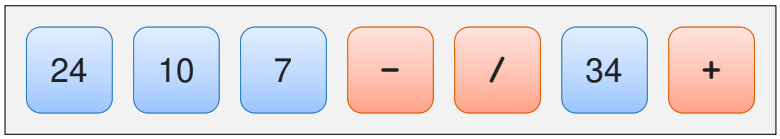
Compute Postfix Pseudo-code

```
while input queue is not empty
   dequeue a token (value or operator)
   if token is a value, push it on the stack
   if token is an operator
      pop two numbers from the stack
      compute the result (using the token)
      push the result on the stack
   end if
end while

...Afterwards, the final result is on the stack
```

Compute Postfix Demo

Input Queue

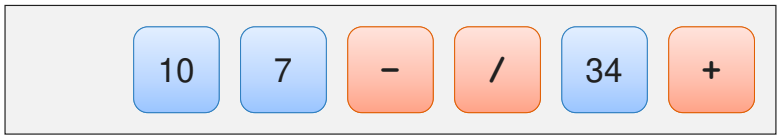


24 / (10 – 7) + 34

Stack

Compute Postfix Demo

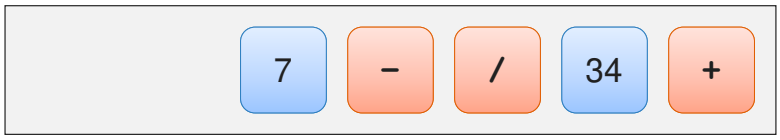
Input Queue



Stack

Compute Postfix Demo

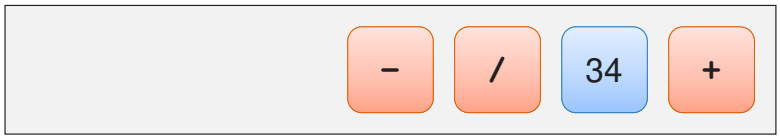
Input Queue



Stack

Compute Postfix Demo

Input Queue



Stack

Compute Postfix Demo

Input Queue

/ 34 +

10 – 7

Stack

24

Compute Postfix Demo

Input Queue

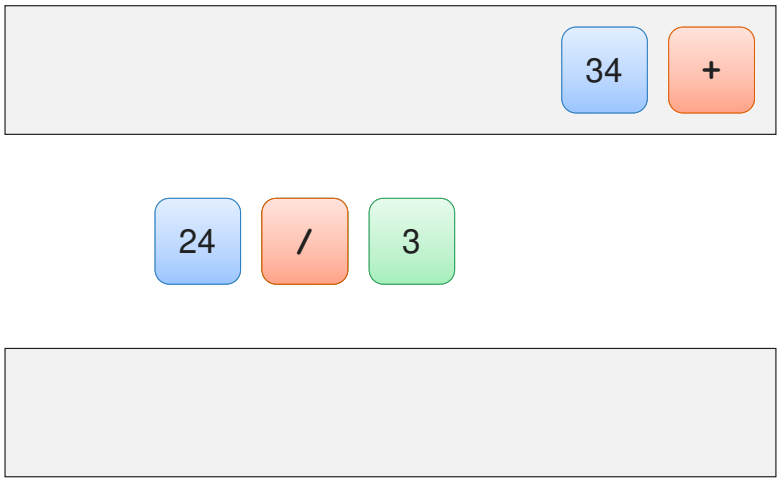
/ 34 +

Stack

24 3

Compute Postfix Demo

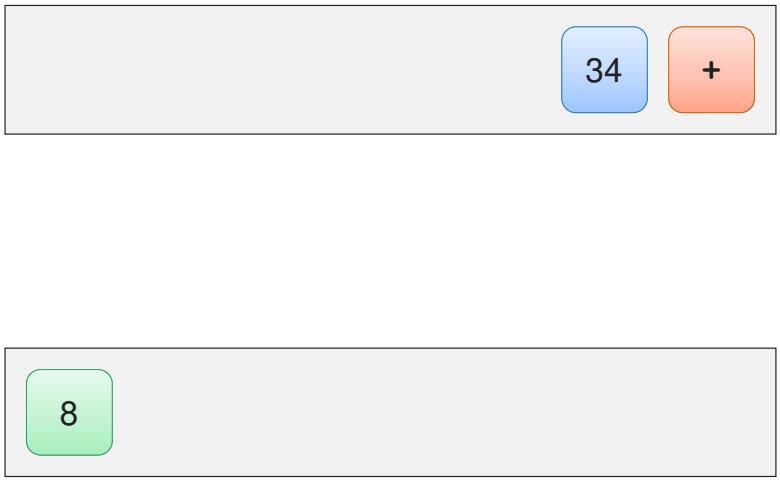
Input Queue



Stack

Compute Postfix Demo

Input Queue



Stack

Compute Postfix Demo

Input Queue

Stack

Compute Postfix Demo

Input Queue

Stack

Compute Postfix Demo

Input Queue

Stack

42

Converting to Prefix or Postfix

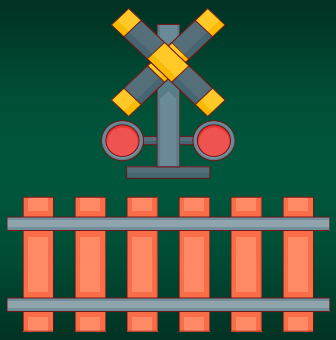
- Why are learning this... *be patient!*
- Converting infix to either postfix or prefix notation is easy to do by hand
- Did you notice that the operands did not change order? They were always *a*, *b*, *c*…
- We just need to rearrange the operators

Convert Infix to Prefix / Postfix

1. Make it a *Fully Parenthesized Expression (FPE)* - one pair of parentheses enclosing each operator and its operands
2. Move the operators to the start (prefix) or end (postfix) of each sub-expression
3. Finally, remove all the parenthesis

Infix to Postfix

```
    a / (b - c) + d
1.  ( ( a / (b - c) ) + d )
2.  ( ( a ( b c - ) / ) d + )
3.  a b c - / d +
```



Infix to Postfix Algorithm

Let the computer do the work…

87

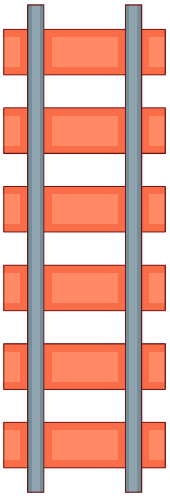
Edsger Dijkstra

- *Edsger Dijkstra* is a World-famous computer scientist
- He invented a wealth of algorithms
- For his contributions, he received the Turing Award

88

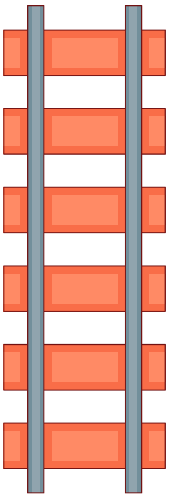
Infix to Postfix Algorithm

- Infix expressions need to be converted to postfix to be evaluated
- Dijkstra's *Shunting-yard algorithm* performs this task

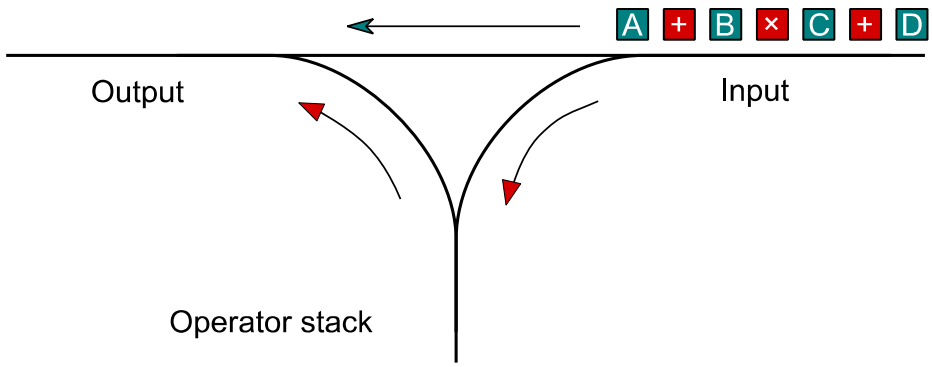


Shunting-yard algorithm

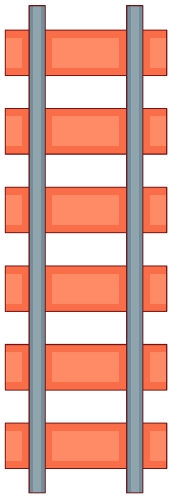
- Named after railroad shunting yards – which move trains onto different tracks
- Dijkstra's solution uses an input queue, operator stack, and output queue



Shunting-yard Algorithm



Shunting-yard Algorithm



- The most basic version of this algorithm requires *Fully-Parenthesized Expression*
- This means, there is no precedence and parenthesis are put around every operator

```
while the input queue has tokens
   read a token from the input queue
   if the token is a…
      operand : add the token to the output queue
      operator : push the token on the stack
      '(' : push it onto the stack
      ')' :
         while the top of stack isn't a '('
            pop an operator from the stack
            add it to the output queue
         end while
         pop and discard the '(' from the stack
   end if
end while
```


FPE Shunting-yard Algorithm

Input Queue: ((a * (b + c)) / d)

Operator Stack:

Output Queue:

FPE Shunting-yard Algorithm

Input Queue: ((a * (b + c)) / d)

Operator Stack:

Output Queue:

FPE Shunting-yard Algorithm

Input Queue: (a * (b + c)) / d)

Operator Stack: (

Output Queue:

FPE Shunting-yard Algorithm

Input Queue: a * (b + c)) / d)

Operator Stack: ((

Output Queue:

FPE Shunting-yard Algorithm

Input Queue: * (b + c)) / d)

Operator Stack: ((

Output Queue: a

FPE Shunting-yard Algorithm

Input Queue: (b + c)) / d)

Operator Stack: ((*

Output Queue: a

FPE Shunting-yard Algorithm

Input Queue: b + c)) / d)

Operator Stack: ((* (

Output Queue: a

FPE Shunting-yard Algorithm

Input Queue: + c)) / d)

Operator Stack: ((* (

Output Queue: a b

FPE Shunting-yard Algorithm

Input Queue: c)) / d)

Operator Stack: ((* (+

Output Queue: a b

FPE Shunting-yard Algorithm

Input Queue:)) / d)

Operator Stack: ((* (+

Output Queue: a b c

FPE Shunting-yard Algorithm

Input Queue:) / d)

Operator Stack: ((* (+)

Output Queue: a b c

FPE Shunting-yard Algorithm

Input Queue:) / d)

Operator Stack: ((*

Output Queue: a b c +

FPE Shunting-yard Algorithm

Input Queue: / d)

Operator Stack: ((*)

Output Queue: a b c +

FPE Shunting-yard Algorithm

Input Queue: / d)

Operator Stack: (

Output Queue: a b c + *

FPE Shunting-yard Algorithm

Input Queue: d)

Operator Stack: (/

Output Queue: a b c + *

FPE Shunting-yard Algorithm

Input Queue:)

Operator Stack: (/

Output Queue: a b c + * d

FPE Shunting-yard Algorithm

Input Queue:

Operator Stack: (/)

Output Queue: a b c + * d

FPE Shunting-yard Algorithm

Input Queue

Operator Stack

Output Queue

FPE Shunting-yard Algorithm

Input Queue

Operator Stack

Output Queue

a b c + * d /

Too Many Paranthesis!

- FPE's are rarely used in real-World examples
- In fact, we use precedence rules to simplify expressions
- Fortunately, the algorithm can be modified, *very easily*, to handle precedence!


```
while the input queue has tokens
   read a token from the input queue
   if the token is a…
      operand : add the token to the  output queue
      operator :  ← NEW RULES! SEE NEXT SLIDE.
      '(' : push it onto the stack
      ')' :
         while the top of stack isn't a '('
            pop an operator from the stack
            add it to the output queue
         end while
         pop and discard the '(' from the stack
   end if
end while
```

Operator: New Rules

```
if operator is left-associative
     while top of stack is ≥ operator and not a '('
         pop the stack
         add it to the output queue
     end while
if operator is right-associative
     while top of stack is > operator and not a '('
         pop the stack
         add it to the output queue
     end while
push the operator onto the stack
```


Operator Associatively

Operator	Associatively
+ − * /	Left
^ (exponent)	Right

Shunting-yard Algorithm Example 1

Input Queue

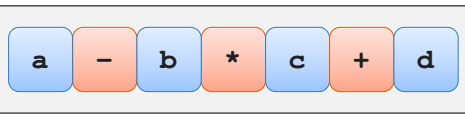
`a - b * c + d`

Operator Stack

Output Queue

Shunting-yard Algorithm Example 1

Input Queue



Operator Stack

Output Queue

Shunting-yard Algorithm Example 1

Input Queue: - b * c + d

Operator Stack:

Output Queue: a

Shunting-yard Algorithm Example 1

Input Queue: b * c + d

Operator Stack: -

Output Queue: a

Shunting-yard Algorithm Example 1

Input Queue: * c + d

Operator Stack: -

Output Queue: a b

Shunting-yard Algorithm Example 1

Input Queue: c + d

Operator Stack: - *

Output Queue: a b

Shunting-yard Algorithm Example 1

Input Queue: + d

Operator Stack: – *

Output Queue: a b c

Shunting-yard Algorithm Example 1

Input Queue: d

Operator Stack: – * +

The precedence of * – are both ≥ than +

Output Queue: a b c

Shunting-yard Algorithm Example 1

Input Queue: d

Operator Stack: +

Output Queue: a b c * -

Shunting-yard Algorithm Example 1

Input Queue:

Operator Stack: +

Remaining stack items pop'd

Output Queue: a b c * - d

Shunting-yard Algorithm Example 1

Input Queue

Operator Stack

Output Queue

a b c * - d +

Shunting-yard Algorithm Example 1

Input Queue

Operator Stack

Output Queue

a b c * - d +

Shunting-yard Algorithm Example 2

Input Queue

`a + (b - c * d) / e - f`

Operator Stack

Output Queue

Shunting-yard Algorithm Example 2

Input Queue

a	+	(	b	-	c	*	d	)	/	e	-	f

Operator Stack

Output Queue

Shunting-yard Algorithm Example 2

Input Queue: + (b - c * d) / e - f

Operator Stack:

Output Queue: a

Shunting-yard Algorithm Example 2

Input Queue: (b - c * d) / e - f

Operator Stack: +

Output Queue: a

Shunting-yard Algorithm Example 2

Input Queue: b − c * d) / e − f

Operator Stack: + (

Output Queue: a

Shunting-yard Algorithm Example 2

Input Queue: − c * d) / e − f

Operator Stack: + (

Output Queue: a b

Shunting-yard Algorithm Example 2

Input Queue: c * d) / e - f

Operator Stack: + (-

Output Queue: a b

Shunting-yard Algorithm Example 2

Input Queue: * d) / e - f

Operator Stack: + (-

Output Queue: a b c

Shunting-yard Algorithm Example 2

Input Queue: d) / e - f

Operator Stack: + (- *

Output Queue: a b c

Shunting-yard Algorithm Example 2

Input Queue:) / e - f

Operator Stack: + (- *

Output Queue: a b c d

Shunting-yard Algorithm Example 2

Input Queue: / e - f

Operator Stack: + (- *)

Output Queue: a b c d

Shunting-yard Algorithm Example 2

Input Queue: / e - f

Operator Stack: +

Output Queue: a b c d * -

Shunting-yard Algorithm Example 2

Input Queue: e − f

Operator Stack: + /

Output Queue: a b c d * −

Shunting-yard Algorithm Example 2

Input Queue: − f

Operator Stack: + /

Output Queue: a b c d * − e

Shunting-yard Algorithm Example 2

Input Queue: f

Operator Stack: + / –

+ / are both ≥ than –

Output Queue: a b c d * – e

Shunting-yard Algorithm Example 2

Input Queue: f

Operator Stack: –

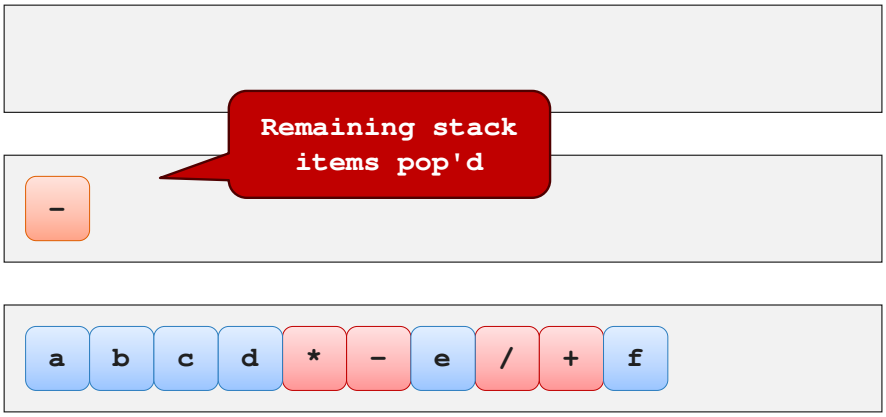
Output Queue: a b c d * – e / +

Shunting-yard Algorithm Example 2

Input Queue

Operator Stack

Output Queue

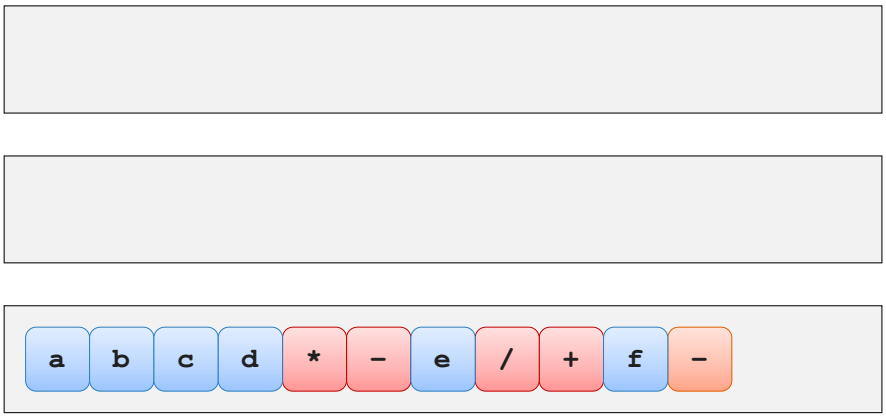


Shunting-yard Algorithm Example 2

Input Queue

Operator Stack

Output Queue



Shunting-yard Algorithm Example 2

Input Queue	
Operator Stack	
Output Queue	a b c d * - e / + f -

Testing Our Result

a + (b - c * d) / e - f

1. ((a + ((b - (c * d)) / e)) - f)

2. ((a ((b (c d *) -) e /) +) f -)

3. a b c d * - e / + f -